
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 5a: Hardware-Accelerated Numerics: ARM Cortex-M33 Single-Precision FPU vs. RP2350 Double-Precision Coprocessor (DCP)

ARM Cortex-M33 • GDB • Ghidra

September 26, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

By the end of this tutorial, you will be able to:

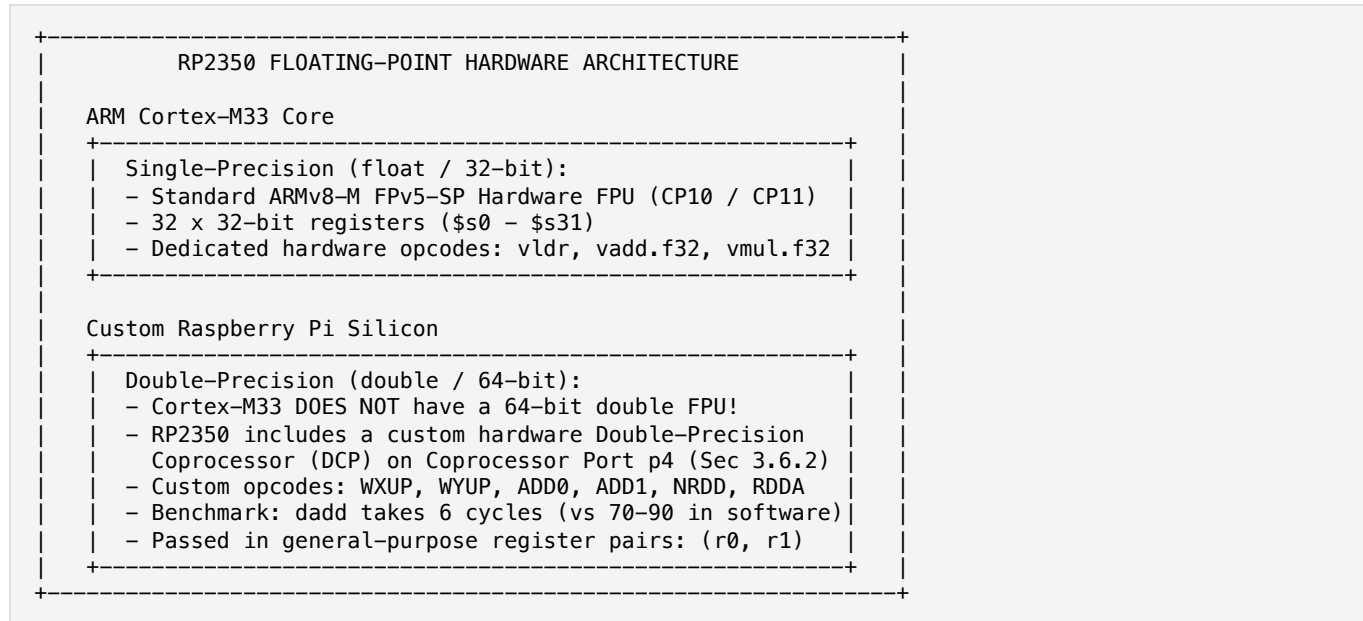
- Understand the hardware split on the RP2350: Single-precision FPU (Cortex-M33 CP10/CP11 hardware) vs. Double-precision Coprocessor (DCP on Coprocessor port `p4`).
- Analyze the RP2350 datasheet (Section 3.6.2 and Section 3.6.4) to discover how Raspberry Pi solved the historic Cortex-M33 double-precision performance bottleneck.
- Understand why double-precision math in standard software emulation takes 70 to 90 clock cycles per operation, and how the RP2350 hardware DCP executes it in just 6 cycles.
- Master the IEEE 754 binary standard from the silicon level: Sign, Biased Exponent, and Fractional Mantissa for both 32-bit single (`float`) and 64-bit double (`double`).
- Deconstruct floating-point constants into exact IEEE 754 hexadecimal bit patterns using `float_hex_converter.py`.
- Navigate Little-Endian byte and word ordering in Flash literal pools and CPU register pairs.
- Understand ARM Cortex-M33 calling conventions (AAPCS): passing single floats in hardware VFP registers (`s0..s31`) vs. passing 64-bit doubles in aligned general-purpose register pairs (`r0 : r1` and `r2 : r3`).
- Identify the C variadic promotion rule that causes `printf()` to automatically promote 32-bit floats to 64-bit doubles at compile time.
- Trace Thumb-2 machine code disassembled with `arm-none-eabi-objdump`, contrasting single-precision FPU instructions (`vldr.32`, `vadd.f32`) against double-precision operations (`ldrd r4, r5`, DCP sequences).
- Debug floating-point operations live over SWD using GDB and an OpenOCD hardware Debug Probe.
- Inspect and manipulate FPU registers in GDB (`info registers float`, `p/f $s0`, `$fpscr`).
- Reconstruct and inspect 64-bit doubles live in GDB from general-purpose register pairs.
- Reverse engineer floating-point literal pools in Ghidra and understand decompiler representations.
- Perform binary patching on IEEE 754 float and double constants directly in raw `.bin` firmware using the bulletproof **Bytes Window** workflow.

Part 1: The Silicon Reality of Floating-Point on RP2350

The Microcontroller Floating-Point Dilemma

In embedded systems, mathematical computations frequently involve real-world physical quantities: temperatures, voltages, GPS coordinates, angular velocities, and sensor calibrations. In pure software, floating-point arithmetic is computationally brutal. A simple 32-bit addition executed via software emulation libraries (`soft-float`) requires dozens of integer bit-shifts, exponent alignments, and normalization loops.

Modern 32-bit microcontrollers address this by incorporating dedicated hardware:



The Single-Precision FPU (FPv5-SP)

The primary processor in the RP2350 is the dual-core **ARM Cortex-M33**. In the RP2350 configuration, ARM's standard single-precision Floating Point Unit (**FPv5-SP-D16-M**) is integrated into each core.

According to **Section 3.6.4** and **Section 3.7.3.5** of the RP2350 datasheet:

- Coprocessor Access:** The FPU is accessed via ARM coprocessor ports **10 and 11**. During microcontroller startup (`SystemInit()`), software must enable access by setting bits 20:23 in the Coprocessor Access Control Register (`CPACR`). If an application executes an FPU instruction before `CPACR.CP10` and `CPACR.CP11` are set, the CPU immediately generates a `NOCPU UsageFault` exception.
- Register File:** The FPU adds thirty-two 32-bit registers named `s0 through s31`. These can also be addressed as sixteen 64-bit double-word views named `d0 through d15`.
- Execution Speed:** Hardware single-precision operations (`vadd.f32`, `vsub.f32`, `vmul.f32`) execute in just **1 to 3 clock cycles**.
- Status Register:** The Floating-Point Status and Control Register (`FPSCR`) records cumulative IEEE 754 condition codes (Invalid Operation, Division by Zero, Overflow, Underflow, Inexact) and controls rounding modes and Flush-to-Zero (`FZ`).

The Double-Precision Challenge

While single-precision (32-bit) floating-point is sufficient for simple temperature readings, high-precision engineering systems demand **64-bit double-precision** (`double`). For example, in GPS satellite navigation:

- A 32-bit `float` has only 23 bits of mantissa, providing roughly **7 decimal digits of precision**. In geographic coordinates, this yields a positional resolution of roughly 1.1 meters at the equator.
- A 64-bit `double` has 52 bits of mantissa, providing **15 to 17 decimal digits of precision**, resolving sub-millimeter positions.

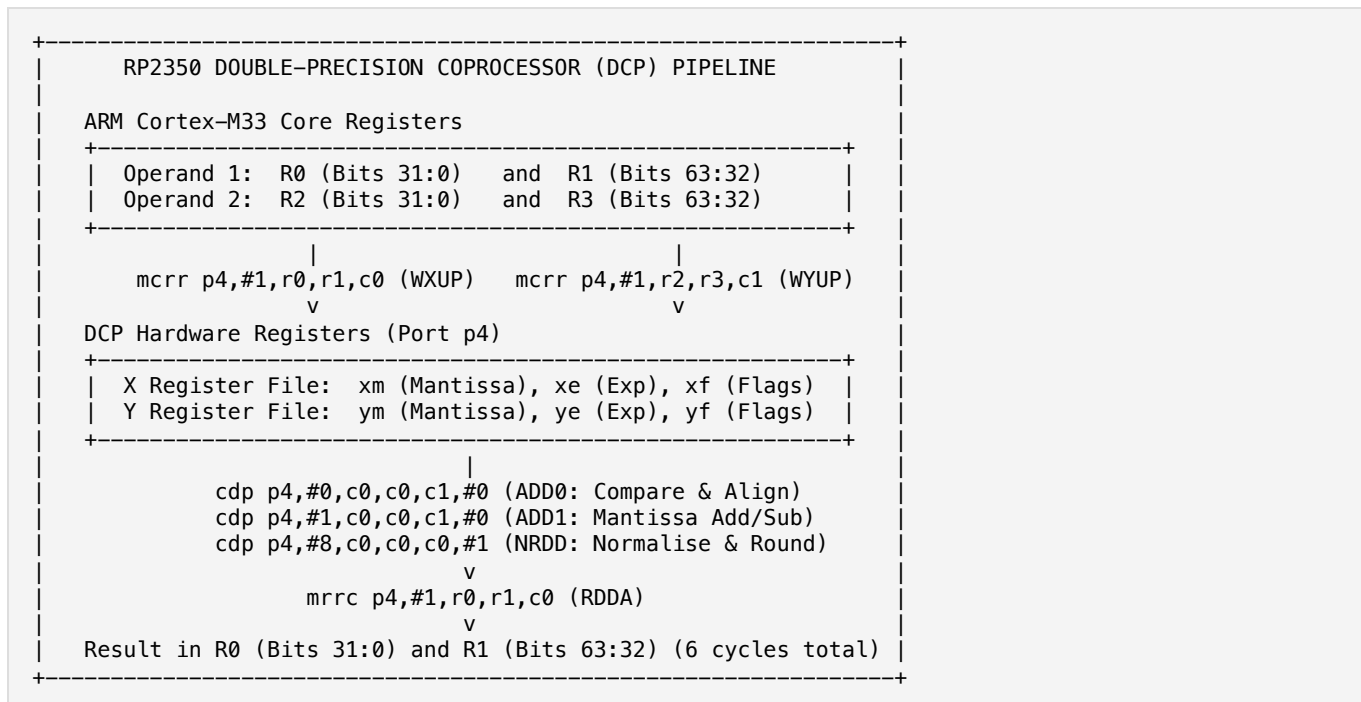
However, the ARM Cortex-M33 **does not include a double-precision hardware FPU**. Adding full 64-bit floating-point units to embedded cores requires enormous silicon surface area, increased leakage current, and complex multi-cycle routing.

In ordinary microcontrollers, any calculation using `double` falls back to software emulation libraries (`__aeabi_dadd`, `__aeabi_dmul`, `__aeabi_ddiv`). A single double-precision addition in software takes **70 to 90 clock cycles**, and a square root can consume over **600 clock cycles**!

The RP2350 Solution: Double-Precision Coprocessor (DCP)

To overcome this bottleneck without bloating the ARM core, Raspberry Pi engineered custom silicon on the RP2350 die: the **Double-Precision Coprocessor (DCP)**, documented in **Section 3.6.2** of the RP2350 datasheet.

The DCP attaches directly to the Cortex-M33's coprocessor bus on **Coprocessor Port p4**.



Datasheet Performance Benchmarks (Table 113)

Section 3.6.2.10 (Table 113) of the RP2350 datasheet documents the dramatic performance difference achieved by the DCP:

Operation	Using RP2350 DCP	Full Hardware FPU	Software-Only Emulation
<code>dadd</code> (Addition)	6 cycles	2 - 6 cycles	70 - 90 cycles

Operation	Using RP2350 DCP	Full Hardware FPU	Software-Only Emulation
<code>dsub</code> (Subtraction)	6 cycles	2 - 6 cycles	70 - 90 cycles
<code>dmul</code> (Multiplication)	17 cycles	3 - 7 cycles	75 - 90 cycles
<code>ddiv</code> (Division)	51 cycles	13 - 60 cycles	135 - 600 cycles
<code>dsqrt</code> (Square Root)	49 cycles	15 - 62 cycles	130 - 650 cycles
<code>dcmp</code> (Compare)	4 cycles	1 - 3 cycles	20 - 40 cycles

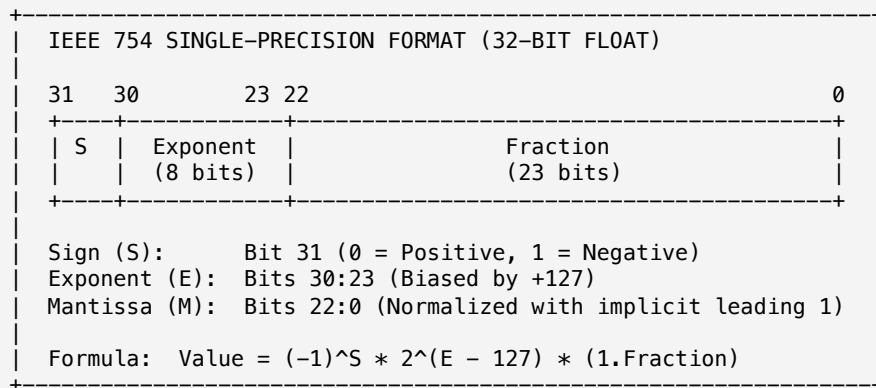
By utilizing the DCP, the RP2350 accelerates double-precision math by **10x to 15x** compared to conventional software emulation, making complex navigation, scientific filtering, and cryptographic computations viable on a low-cost microcontroller.

Part 2: IEEE 754 Binary Anatomy (Single vs. Double Precision)

To reverse engineer or patch floating-point numbers in firmware, you must be able to read and construct raw IEEE 754 bytes directly.

Single-Precision (32-bit float)

A 32-bit float conforms to the IEEE 754 single-precision format:



Step-by-Step Conversion: 42.5f

Let's convert 42.5 to raw IEEE 754 single-precision hex:

1. Separate whole and fractional parts:

$$42_{10} = 32 + 8 + 2 = 101010_2$$

$$0.5_{10} = 2^{-1} = 0.1_2$$

$$42.5_{10} = 101010.1_2$$

2. Normalize into scientific notation ($1.mimes2^n$):

$$101010.1_2 = 1.010101_2 \times 2^5$$

3. Calculate the biased exponent:

$$E = n + \text{bias} = 5 + 127 = 132 = 10000100_2 = 0x84$$

4. Extract the 23-bit mantissa (drop the implicit leading 1):

$$M = 01010100000000000000000_2$$

5. Assemble the 32 bits:

- Sign (1 bit): 0
- Exponent (8 bits): 10000100
- Mantissa (23 bits): 01010100000000000000000
- Binary string: 0100 0010 0010 1010 0000 0000 0000 0000
- Hexadecimal: 0x422A0000

Double-Precision (64-bit double)

A 64-bit double expands both range and precision dramatically:

IEEE 754 DOUBLE-PRECISION FORMAT (64-BIT DOUBLE)			
63	62	52 51	0
+-----+-----+-----+-----+			
S	Exponent	Fraction	
	(11 bits)	(52 bits)	
+-----+-----+-----+-----+			
Sign (S): Bit 63 (0 = Positive, 1 = Negative)			
Exponent (E): Bits 62:52 (Biased by +1023)			
Mantissa (M): Bits 51:0 (Normalized with implicit leading 1)			
Formula: Value = (-1)^S * 2^(E - 1023) * (1.Fraction)			

Step-by-Step Conversion: 42.52525

Let's convert 42.52525 (the constant used in `0x0011_double-floating-point-data-type`):

1. **Integer part:** $42 = 101010_2$
2. **Fractional part:** $0.52525 \times 2 = 1.0505$ (bit 1), $0.0505 \times 2 = 0.101$ (bit 0), etc. Normalized binary:

$$1.0101010000110011101101100100010110100001110010101100_2 \times 2^5$$

3. **Calculate biased exponent:**

$$E = 5 + 1023 = 1028 = 10000000100_2 = 0x404$$

4. **Assemble the 64 bits:**

- Sign (1 bit): 0
- Exponent (11 bits): 100 0000 0100
- Mantissa (52 bits): 0101 0100 0011 0011 1011 0110 0100 0101 1010 0001 1100 1010 1100
- Hexadecimal: 0x4045433B645A1CAC

Little-Endian Ordering in Memory and Literal Pools

The RP2350 Cortex-M33 is configured in **Little-Endian** byte order. In 64-bit values, both the bytes within each 32-bit word and the words themselves are stored least-significant first:

LITTLE-ENDIAN STORAGE OF 64-BIT DOUBLE IN FLASH								
Double Value: 42.52525								
Full 64-bit Hex: 0x4045433B645A1CAC								
Lower 32-bit Word (Bits 31:0): 0x645A1CAC								
Upper 32-bit Word (Bits 63:32): 0x4045433B								
Memory Offset:	+0	+1	+2	+3	+4	+5	+6	+7
Flash Bytes:	AC	1C	5A	64	3B	43	45	40
	Lower Word (R4)				Upper Word (R5)			

When reversing or patching in Ghidra, notice that the lower word (0x645A1CAC) precedes the upper word (0x4045433B) in memory. If you mistakenly patch only the lower word, you only alter the least significant decimal digits. If you patch the upper word, you modify the sign, exponent, and highest-order mantissa bits!

Part 3: The C Compiler, ABI, and Register Conventions

Understanding how compilers generate machine code for floating-point operations is vital for both GDB debugging and Ghidra reverse engineering.

AAPCS-VFP: Hardware Float Register Passing

The ARM Architecture Procedure Call Standard (AAPCS) defines how parameters are passed to functions:

ARM REGISTER MARSHALLING: SINGLE VS. DOUBLE PRECISION

Single-Precision Function: `void process_float(float val);`

Parameter passed in hardware FPU register:	<code>\$s0</code>
Additional float arguments passed in:	<code>\$s1, \$s2, \$s3</code>

Double-Precision Function: `void process_double(double val);`

Parameter passed in 64-bit register pair:	<code>(R0, R1)</code>
R0 holds lower 32 bits (mantissa bits 31:0)	
R1 holds upper 32 bits (sign, exponent, mantissa 51:32)	
Additional double argument passed in:	<code>(R2, R3)</code>

The Variadic Promotion Rule (`printf`)

A major source of confusion in reverse engineering is why a program that declares a 32-bit `float` suddenly compiles with 64-bit double operations:

```
float fav_num = 42.5;
printf("fav_num: %f\r\n", fav_num);
```

Under the **ISO C standard**, functions with variable argument lists (such as `printf`) apply **default argument promotions**:

- Any integer type narrower than `int` is promoted to `int`.
- Any `float` is automatically promoted to `double`!

In the disassembly of `0x000e_floating-point-data-type`, the compiler never passes a 32-bit float to `printf`. Instead, it converts `42.5f` (`0x422A0000`) into `42.5 double` (`0x4045400000000000`):

```
1000023a: 2400      movs    r4, #0           @ Lower 32 bits = 0
1000023c: 4d03      ldr     r5, [pc, #12]    @ Upper 32 bits = 0x40454000
1000023e: 4622      mov     r2, r4           @ Marshal lower word into R2
10000240: 462b      mov     r3, r5           @ Marshal upper word into R3
10000242: 4803      ldr     r0, [pc, #12]    @ R0 = format string pointer
10000244: f002 ff52 bl      100030ec         @ Call __wrap_printf
```

Notice:

1. `R0` holds the format string address.
2. `R1` is skipped because the AAPCS requires 64-bit double arguments to be **8-byte aligned**, which restricts them to even-odd register pairs: `(R0, R1)` or `(R2, R3)`.
3. `R2` receives the lower 32 bits (`0x00000000`).
4. `R3` receives the upper 32 bits (`0x40454000`).

Part 4: Hands-On Live Lab (The Practical Proof)

Let's examine a live embedded firmware project that combines single-precision sensor calibration with double-precision navigation thresholding.

Source Code: telemetry_node.c

```

#include <stdio.h>
#include <stdbool.h>
#include "pico/stdlib.h"

// Single-precision calibration factor (stored in Flash literal pool)
static const float CALIBRATION_GAIN = 1.045f; // 0x3F85C28F

// Double-precision coordinate boundary (stored in Flash literal pool)
static const double MAX_LATITUDE = 37.7749; // 0x4042E33020C49BA6

float filter_sensor(float raw) {
    return raw * CALIBRATION_GAIN;
}

bool evaluate_geofence(double current_lat) {
    if (current_lat > MAX_LATITUDE) {
        return false; // Out of bounds
    }
    return true; // Safe
}

int main(void) {
    stdio_init_all();
    float voltage = 3.3f;
    double current_lat = 37.7750; // Dangerously out of bounds!

    while (true) {
        float calibrated = filter_sensor(voltage);
        bool status = evaluate_geofence(current_lat);
        printf("V: %f | Safe: %d\r\n", (double)calibrated, (int)status);
        sleep_ms(1000);
    }
}

```

Disassembly Analysis: Single vs. Double Precision

When compiled with `arm-none-eabi-gcc -O2 -mcpu=cortex-m33 -mfloat-abi=hard -mfpu=fpv5-sp-d16`, look at the stark contrast in machine instructions:

1. Single-Precision Assembly (filter_sensor):

```

10000250 <filter_sensor>:
10000250: ed9f 7a02    vldr     s15, [pc, #8]    @ Load CALIBRATION_GAIN (0x3F85C28F)
10000254: ee20 0a27    vmul.f32 s0, s0, s15     @ Hardware FPU single-cycle multiply
10000258: 4770                bx      lr               @ Return, result in $s0
1000025a: bf00                nop
1000025c: 3f85c28f    .word   0x3f85c28f      @ Literal pool constant

```

- `vldr s15, [pc, #8]` : Loads directly from Flash into hardware FPU register `$s15`.
- `vmul.f32 s0, s0, s15` : Executes in hardware in 1 clock cycle.
- The input parameter arrives in `$s0`, and the return value exits in `$s0`.

2. Double-Precision Assembly (`evaluate_geofence`):

```

10000260 <evaluate_geofence>:
10000260: b508      push    {r3, lr}
10000262: 4a03      ldr     r2, [pc, #12]    @ Load lower word: 0x20C49BA6
10000264: 4b04      ldr     r3, [pc, #16]    @ Load upper word: 0x4042E330
10000266: f000 f810 bl      __aeabi_cdcmple @ Call DCP compare accelerator
1000026a: bf38      it      cc
1000026c: 2000      movcc   r0, #0          @ 0 if out of bounds
1000026e: bd08      pop     {r3, pc}
10000270: 20c49ba6 .word    0x20c49ba6      @ Lower 32 bits of MAX_LATITUDE
10000274: 4042e330 .word    0x4042e330      @ Upper 32 bits of MAX_LATITUDE

```

- The double arrives in `R0` (lower 32 bits) and `R1` (upper 32 bits).
- The threshold is loaded from Flash literal pool into `R2` and `R3`.
- The function calls `__aeabi_cdcmple`, which executes the custom `cdp p4` hardware instructions on the RP2350 Double-Precision Coprocessor.

Part 5: Live Dynamic Debugging with GDB over OpenOCD

Now let's connect GDB to the live RP2350 microcontroller and inspect both single and double-precision math.

Launching the Debug Session

Terminal 1 (OpenOCD):

```
openocd -f interface/cmsis-dap.cfg -f target/rp2350.cfg
```

Terminal 2 (GDB):

```

arm-none-eabi-gdb build/telemetry_node.elf
(gdb) target extended-remote :3333
(gdb) monitor reset halt

```

1. Inspecting Hardware FPU Registers

Set a breakpoint at `filter_sensor`:

```

(gdb) hb filter_sensor
(gdb) c
Continuing.
Breakpoint 1, filter_sensor (raw=3.3) at telemetry_node.c:12

```

Inspect the FPU register state:

```
(gdb) info registers float
```

GDB prints the full 32-register single-precision register file:

s0	3.30000019	(raw = 0x40533333)
s1	0	(raw = 0x00000000)
...		
s15	1.04499996	(raw = 0x3f85c28f)
fpscr	0	[]

Notice:

- s0 holds the input argument (3.3f pprox 0x40533333).
- Single-precision float precision limit: 3.3 cannot be represented with infinite precision in binary, resulting in 3.30000019 .

Step one instruction past the multiplication:

```
(gdb) stepi
(gdb) p/f $s0
$1 = 3.44850016
```

Live FPU Register Hijacking:

Force a new calibration gain directly inside the FPU hardware:

```
(gdb) set $s0 = 10.0
(gdb) p/f $s0
$2 = 10
```

2. Inspecting and Reconstructing 64-bit Doubles in GDB

Now continue to evaluate_geofence :

```
(gdb) hb evaluate_geofence
(gdb) c
Continuing.
Breakpoint 2, evaluate_geofence (current_lat=37.7750) at telemetry_node.c:16
```

Because double-precision values live in general-purpose register pairs on Cortex-M33, info registers float will **not** show current_lat !

Instead, examine R0 and R1 :

```
(gdb) info registers r0 r1
r0          0x851eb852      -2061617070
r1          0x4042e333      1078125363
```

How do we interpret r0 and r1 as a double in GDB? Use GDB's memory casting syntax by treating the register address as a pointer to double :

```
(gdb) p *(double*)&($r0)
$3 = 37.775000000000006
```

Or print the literal pool threshold at address 0x10000270 :

```
(gdb) x/1fg 0x10000270
0x10000270:    37.774900000000001
```

Notice the command `x/1fg`:

- `x`: Examine memory.
- `1`: One unit.
- `f`: Format as floating-point.
- `g`: Giant-word (8 bytes / 64 bits)!

If you examine as words with `x/2xw`:

```
(gdb) x/2xw 0x10000270
0x10000270:    0x20c49ba6    0x4042e330
```

- First word: `0x20C49BA6` (least significant 32 bits)
- Second word: `0x4042E330` (most significant 32 bits)

Live Double Register Hijacking:

To force `evaluate_geofence` to evaluate the coordinate as safe without modifying code, modify `R1` (the upper word containing the exponent and sign):

```
(gdb) set $r1 = 0x40400000
(gdb) p *(double*)(&($r0))
$4 = 32.0000000019231
```

The coordinate is now 32.0° , which is less than 37.7749° . The geofence check passes!

Part 6: Static Analysis and Binary Patching in Ghidra

In reverse engineering real firmware images, you won't have source code or symbol tables. You will analyze a stripped `.bin` file in Ghidra.

1. Locating Floating-Point Literals in Ghidra

When you import `telemetry_node.bin` at Flash base address `0x10000000`:

1. Ghidra decodes Thumb-2 instructions. In `filter_sensor`, it identifies:

```
10000250 ed 9f 7a 02    vldr.32    s15, [PC, #0x8]
10000254 ee 20 0a 27    vmul.f32   s0, s0, s15
```

2. In the Decompiler panel, Ghidra detects the floating-point multiplication:

```
float filter_sensor(float param_1) {
    return param_1 * 1.04500003;
}
```

3. In `evaluate_geofence`, Ghidra decodes:

```
10000262 4a 03          ldr        r2, [DAT_10000270]
10000264 4b 04          ldr        r3, [DAT_10000274]
```

At `0x10000270`, right-click the bytes and select **Data** -> **double**. Ghidra formats the 8 bytes directly into:

```
10000270 a6 9b c4 20    double    37.774900000000000
          30 e3 42 40
```

2. Binary Patching Floating-Point Literals: The Bytes Window Workflow

Suppose our mission is to patch the geofence threshold from `37.7749` to `45.0` directly inside the binary image so that higher coordinates are accepted permanently.

Step 1: Calculate the Target IEEE 754 Bytes

Run `float_hex_converter.py`:

```
python3 WEEK05/float_hex_converter.py 45.0
```

Output:

```
Value: 45.0
Single (32-bit): 0x42340000
Double (64-bit): 0x4046800000000000
  Lower 32-bit Word: 0x00000000
  Upper 32-bit Word: 0x40468000
  Little-Endian Bytes: 00 00 00 00 00 80 46 40
```

Step 2: Why Right-Click "Patch Instruction" Fails on Literal Pools

If you attempt to right-click on the `ldr` instructions in Ghidra and change their target operand, Ghidra's assembler will fail or generate an invalid PC-relative offset that corrupts nearby code. In ARM Thumb-2, **you must patch the literal pool data itself, not the load instruction!**

Step 3: The Bulletproof Bytes Window Workflow

GHIDRA BYTES WINDOW FLOAT PATCHING WORKFLOW

```

Step 1: Open Bytes Window
Window -> Bytes: telemetry_node.bin

Step 2: Enable Edit Mode
Click the Pencil Icon in the Bytes Window toolbar

Step 3: Clear Code/Data Bytes in Listing Window
Click address 0x10000270 in Listing Window -> Press 'C'
(Prevents type locks while modifying raw memory)

Step 4: Edit Bytes in Bytes Window (Little-Endian Order)
Original: A6 9B C4 20 30 E3 42 40  (37.7749)
Patch to: 00 00 00 00 00 80 46 40  (45.0)

Step 5: Re-disassemble / Re-type in Listing Window
Click 0x10000270 in Listing -> Press 'D' (or Data -> double)
Decompiler immediately updates: if (current_lat > 45.0)

```

1. Ensure the Bytes window is open (**Window -> Bytes**).
2. Click the **Pencil Icon (Toggle Edit Mode)**.
3. In the Listing window, click address `0x10000270` and press **C (Clear Code Bytes)**.
4. In the Bytes window at offset `10000270`, replace the 8 bytes:

```

Old: A6 9B C4 20 30 E3 42 40
New: 00 00 00 00 00 80 46 40

```

5. Click back in the Listing window at `0x10000270`, right-click and select **Data -> double**.
6. Inspect the Decompiler window: `evaluate_geofence` now compares against `45.0`!
7. Export the modified image via **File -> Export Program...** as `telemetry_node_patched.bin`.

Part 7: Summary & Quick Reference Sheet

Hardware Floating-Point Architecture Comparison

Feature	Single-Precision (float)	Double-Precision (double)
Silicon Implementation	ARM Cortex-M33 Fpv5-SP Core	Custom Silicon DCP Coprocessor
Coprocessor Port	CP10 / CP11	Port p4
Width	32 bits (1 sign, 8 exp, 23 mantissa)	64 bits (1 sign, 11 exp, 52 mantissa)
Exponent Bias	+127	+1023
Decimal Precision	pprox 7 significant decimal digits	pprox 15 - 17 significant decimal digits
Primary Registers	\$s0 through \$s31\$	Register pairs (R0, R1) or (R2, R3)

Feature	Single-Precision (float)	Double-Precision (double)
Key Assembly Instructions	vldr.f32 , vadd.f32 , vmul.f32 , vcmp.f32	ldrd r4, r5 , cdp p4 , mcrr p4 , mrrc p4
Typical Operation Cycles	1 to 3 cycles	6 cycles (dadd), 17 cycles (dmul)

GDB Quick Reference for Numerics

Command	Action
info registers float	Display all 32 single-precision FPU registers and FPSCR
p/f \$s0	Print FPU register \$s0 as a formatted floating-point number
set \$s0 = 42.5	Override FPU register \$s0 at runtime
x/1fw <addr>	Examine 32-bit word in memory as IEEE 754 float
x/1fg <addr>	Examine 64-bit double-word in memory as IEEE 754 double
x/2xw <addr>	Examine 64-bit double in memory as two 32-bit hex words
p *(double*)&(\$r0)	Reconstruct a 64-bit double from register pair \$r0:\$r1
set \$r1 = 0x4042e330	Modify upper 32 bits (sign/exponent) of a double in register pair